

Complex Event Recognition under Time Constraints: Towards a Formal Framework for Efficient Query Evaluation

JULIÁN GARCÍA, Pontificia Universidad Católica de Chile and IMFD, Chile

CRISTIAN RIVEROS, Pontificia Universidad Católica de Chile and IMFD, Chile

Complex Event Recognition (CER) establishes a relevant solution for processing streams of events, giving users timely information. CER systems detect patterns in real-time, producing complex events and generating responses to them. In these tasks, time is a first-class citizen. Indeed, the time-sequence model distinguishes CER from other solutions, like data stream management systems. Surprisingly, until now, time constraints are usually included in CER query languages and models in a restricted way, and we still lack an understanding of the expressiveness and of efficient algorithms concerning this crucial feature: time.

This work studies CER under time constraints regarding its query language, computational models, and streaming evaluation algorithms. We start by introducing an extension of Complex Event Logic (CEL), called timed CEL, with simple time operators. We show that timed CEL aids in modeling CER query languages in practice, serving as a proxy to study the expressive power of such languages under time constraints. For this purpose, we introduce an automata model for studying timed CEL, called timed Complex Event Automata (timed CEA). This model extends the existing CEA model with clocks, combining CEA and timed automata in a single model. We show that timed CEL and timed CEA are equally expressive, giving the first characterization of CER query languages under time constraints. Then, we move towards understanding the efficient evaluation of timed CEA over streams concerning its determinization and efficient algorithms. We present a class of timed CEA that are closed under determinization; furthermore, we show that this class contains swg-queries, an expressive class of CER queries recently introduced by Kleest-Meißner et al. Finally, we present a streaming evaluation algorithm with constant update time and output-linear delay for evaluating deterministic monotonic timed CEA with a single clock, which have only less equal or greater equal comparisons.

CCS Concepts: • **Theory of computation** → **Timed and hybrid models; Formal languages and automata theory**.

Additional Key Words and Phrases: Query evaluation, streams, complex event recognition, automata theory, timed automata.

ACM Reference Format:

Julián García and Cristian Riveros. 2025. Complex Event Recognition under Time Constraints: Towards a Formal Framework for Efficient Query Evaluation. *Proc. ACM Manag. Data* 3, 2 (PODS), Article 94 (May 2025), 17 pages. <https://doi.org/10.1145/3725231>

1 Introduction

Complex Event Processing (CEP) forms a set of solutions and technologies for processing data streams in real-time [14, 19]. CEP systems model their data sources as an infinite sequence of real-time events coming, for example, from sensor measurements, traffic reports, weather events, or social media messages. Complex Event Recognition (CER) is a subtask in CEP that processes event

Authors' Contact Information: Julián García, Pontificia Universidad Católica de Chile and IMFD, Santiago, Chile, jgarcg@uc.cl; Cristian Riveros, Pontificia Universidad Católica de Chile and IMFD, Santiago, Chile, cristian.riveros@uc.cl.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 2836-6573/2025/5-ART94
<https://doi.org/10.1145/3725231>

streams in real-time by detecting patterns, producing complex events, and generating responses to them as the events arrive. Users define these patterns through a structured query language that allows them to specify situations of interest in the real world.

Since data represents real-world events, *time* is a first-class citizen in CER. As stated in [14, 36], the time-sequence model is what distinguishes CER from data stream management systems, both in terms of query languages and evaluation algorithms. As early as the first CER systems [28], their query languages recognized the importance of time, incorporating special operators to include time constraints. Indeed, all CER systems include the *WITHIN* clause to specify a time window where the pattern must appear. In other words, supporting time restrictions is a crucial feature of any CER query language today.

Surprisingly, until now, time constraints are usually included in CER query languages and models in a restricted way [14]. As a proof of fact, the *WITHIN* clause is included in all CER query languages but only for specifying a restriction over the overall pattern. In particular, one cannot use this operator internally or nested it inside a pattern. Although there are some proposals for more advanced time operators in CER query languages [19, 26], there is no formal logic or computational model that helps us understand the expressiveness and efficient algorithms concerning this crucial feature: time.

This paper studies CER under time constraints regarding its query language, computational model, and streaming evaluation algorithms. Our first goal is to find a logic that serves as a proxy to study the expressive power of CER query languages under time constraints (Section 3). We propose to extend Complex Event Logic (CEL) with time windows and time operators, which we call *timed CEL*. We show that timed CEL aids in modeling CER query languages, including most operators used in practice for declaring time constraints.

To understand the expressiveness of timed CEL, we look for the right automata model to express time constraints in CER (Section 4). Timed automata is an extension of finite state automata with clocks that has a rich theory and has been successful for formal verification of real-time systems. Our strategy is simple: to bridge the theory of timed automata with CER. We introduce *timed Complex Event Automata* (timed CEA), a model that extends the existing CER automata (called CEA) with clocks, combining CEA and timed automata in a single model. We show that timed CEA is the right model for CER by showing that timed CEL and timed CEA are equally expressive. To the best of our knowledge, this is the first characterization of the expressive power of CER query languages under time constraints.

With a logic and automata model for processing CER queries with time constraints, we move towards finding efficient evaluation algorithms for them. We start by studying the determinization of timed CEA, namely, to convert a timed CEA into a deterministic one (Section 5). This problem is crucial for evaluation since efficient enumeration algorithms over automata models usually require determinizing them to remove duplicates. Unfortunately, not all timed CEA admit an equivalent deterministic one, and thus, we look for classes of timed CEA that we can determinize efficiently. On this line, we present the class of *synchronous resets* timed CEA, which is simple and easy to determinize. Furthermore, we show that this class contains swg-queries, an expressive class of CER queries recently introduced by Kleest-Meißner et al [24].

Having a class of deterministic timed CEA, we study algorithms to evaluate them (Section 6). On this line, we present a streaming evaluation algorithm with constant update time and output-linear delay for evaluating monotonic deterministic timed CEA with a single clock, which have only less equal or greater equal comparisons.

Related work. Here, we do a general review of the relation of this work to CER and timed automata. We discuss more connections throughout the paper whenever they are more suitable.

As stated above, time is a special feature in CER, and all CER systems include event timestamps in their models. Windows is the most common operator for declaring time constraints in query languages [14, 19]. Further, people have proposed efficient algorithms for evaluating queries over a sliding window [11, 35]. Other proposals used timed restrictions between pairs of events [24, 26]. Nevertheless, we are not aware of a work in CER that has studied the expressiveness of such query languages, or efficient algorithms for evaluating queries with general time constraints.

Timed automata [3] have a rich theory and applications in formal verification. This work is based on the theory of timed automata making a bridge with CER and, more generally, with data stream management systems. Logic for timed automata and real-time systems has been proposed in the past [5, 25]. However, these logics are designed for formal verification of systems, rather than for streaming processing. In particular, they have not been used for CER: they are not defined to produce outputs (i.e., complex events) or studied for efficient evaluation over streams. Some works [10, 12] have used timed automata in CER; however, they did not study the same research questions as this work regarding expressive analysis, and efficient query answering.

2 Preliminaries

Sets and intervals. Given a set A , we denote by 2^A the *powerset* of A . We denote by $\mathbb{N}$ the natural numbers. Given $n, m \in \mathbb{N}$ with $n \leq m$, we denote by $[n]$ the set $\{1, \dots, n\}$ and by $[n..m]$ the interval $\{n, n+1, \dots, m\}$ over $\mathbb{N}$. We denote by $\mathbb{Q}_{\geq 0}$ the non-negative rational numbers, and by $\mathbb{Q}_{\geq 0}^{\infty}$ the set $\mathbb{Q}_{\geq 0} \cup \{\infty\}$ where $q < \infty$ for every $q \in \mathbb{Q}_{\geq 0}$. A subset $I \subseteq \mathbb{Q}_{\geq 0}$ is an *interval* over $\mathbb{Q}_{\geq 0}$ if for every $p, q \in I$ and $r \in \mathbb{Q}_{\geq 0}$, if $p \leq r \leq q$, then $r \in I$. We will use the standard notation for specifying intervals over $\mathbb{Q}_{\geq 0}$, namely, as pairs in $\mathbb{Q}_{\geq 0} \times \mathbb{Q}_{\geq 0}^{\infty}$ with squared or circled brackets. For example, $(\frac{1}{2}, 3] = \{p \in \mathbb{Q}_{\geq 0} \mid \frac{1}{2} < p \leq 3\}$, $(1, 2) = \{p \in \mathbb{Q}_{\geq 0} \mid 1 < p < 2\}$, and $[7, \infty) = \{p \in \mathbb{Q}_{\geq 0} \mid 7 \leq p < \infty\}$ are intervals over $\mathbb{Q}_{\geq 0}$ under this notation.

Events and streams. We fix a set T of *event types*, a set A of *attributes names*, and a set D of *data values* (e.g., integers, floats, strings). An *event* (or primitive event) e is a partial mapping $e : A \rightarrow D$ that maps attributes names in A to data values in D . We denote $\text{att}(e)$ the domain of e , called the *attributes of e* , and we assume that $\text{att}(e)$ is finite. We denote by $e(A)$ the data value of the attribute $A \in A$ whenever $A \in \text{att}(e)$. Further, each event e has a *type* in T denoted by $\text{type}(e)$. We define the *size* $|e|$ of an event e as the number of RAM registers to encode e . We write E to denote the *set of all events* over event types T , attributes names A , and data values D . A *stream* is an (arbitrary long) sequence $\bar{S} = e_1 e_2 \dots e_n$ of events where $|\bar{S}| = n$ is the length of the stream.

Example 2.1. As a running example (from [22, Section 3]), suppose the scenario of a national park sensor network measuring the temperature and humidity of the park. For the sake of simplification, assume that there are two types of events: T and H . A *temperature event* e has $\text{type}(e) = T$ and $\text{att}(e) = \{\text{temp}\}$ where $e(\text{temp})$ contains the temperature value in degrees Celsius (e.g., $e(\text{temp}) = 20^\circ$). A *humidity event* e has $\text{type}(e) = H$ and $\text{att}(e) = \{\text{hum}\}$ where $e(\text{hum})$ contains the value as a percentage (e.g., $e(\text{hum}) = 40\%$). In Figure 1, we display a stream $\bar{S}_0$ for this scenario.

Complex events. Fix a finite set X of *variables* and assume that $T \subseteq X$, where T is the set of event types as defined earlier. Let $\bar{S}$ be a stream of length n . A *complex event*¹ of $\bar{S}$ is a triple (i, j, μ) where $i, j \in [n]$, $i \leq j$, and $\mu : X \rightarrow 2^{[i..j]}$ is a function with finite domain. Intuitively, i and j marks the beginning and end of the interval where the complex event happens, and μ stores the events in the interval $[i..j]$ that fired the complex event. In the following, we will usually use C to denote a complex event (i, j, μ) of $\bar{S}$ and omit $\bar{S}$ if the stream is clear from the context. We will

¹In [11, 22], triples of the form (i, j, μ) are called *valuations* and complex events are triples (i, j, D) where $D \subseteq [i..j]$. In this work, we prefer to call (i, j, μ) as complex events to simplify the presentation.

type	<i>H</i>	<i>T</i>	<i>H</i>	<i>H</i>	<i>T</i>	<i>T</i>	<i>T</i>	<i>H</i>	<i>H</i>	...
<i>temp</i>		45°			40°	42°	25°			
<i>hum</i>	25%		20%	25%				70%	18%	...
time (sec)	1.2	1.33	2.5	3.7	4.5	5.3	5.9	6.1	7.2	...
	1	2	3	4	5	6	7	8	9	...

Fig. 1. A timed stream $\bar{S}_0$ of temperatures (*T*) and humidities (*H*) with attributes *temp* (degrees Celcius) and *hum* (percentage), respectively. The timestamp is the third line, measured in seconds. The fourth line is the index position in the stream.

use $\text{interval}(C)$, $\text{start}(C)$, and $\text{end}(C)$ to denote the interval $[i..j]$, the start i , and the end j of C , respectively. Further, by some abuse of notation we will also use $C(X)$ for $X \in \mathbf{X}$ to denote the set $\mu(X)$ of C .

Example 2.2. Complex events of the stream $\bar{S}_0$ of Figure 1 can be $C_1 = (5, 9, \{X \mapsto \{5\}, Y \mapsto \{9\}, H \mapsto \emptyset, T \mapsto \emptyset\})$ and $C_2 = (2, 9, \{X \mapsto \{2\}, Y \mapsto \{9\}, H \mapsto \emptyset, T \mapsto \emptyset\})$ where X and Y are variables in $\mathbf{X}$.

The following operations on complex events will be useful throughout the paper. We define the *union* of complex events C_1 and C_2 , denoted by $C_1 \cup C_2$, as the complex event C' such that $\text{start}(C') = \min\{\text{start}(C_1), \text{start}(C_2)\}$, $\text{end}(C') = \max\{\text{end}(C_1), \text{end}(C_2)\}$, and $C'(X) = C_1(X) \cup C_2(X)$ for every $X \in \mathbf{X}$. Further, we define the *projection over L* of a complex event C , denoted by $\pi_L(C)$, as the complex event C' such that $\text{interval}(C') = \text{interval}(C)$ and $C'(X) = C(X)$ whenever $X \in L$, and $C'(X) = \emptyset$, otherwise. Finally, we denote by (i, j, μ_0) the complex event with trivial mapping μ_0 such that $\mu_0(X) = \emptyset$ for every $X \in \mathbf{X}$.

Predicates of events. A *predicate* is a possibly infinite set P of events. For instance (see Example 2.1), P could be the set of all T -events e such that $e(\text{temp}) \leq 20^\circ$. In our examples, we will use the notation $A \sim a$ where $A \in \mathbf{A}$ and $a \in \mathbf{D}$ to denote the predicate $P = \{e \mid e(A) \sim a\}$ (e.g., $\text{temp} \leq 20^\circ$). We say that an event e satisfies predicate P , denoted $e \models P$, if, and only if, $e \in P$. We generalize this notation from events to a set of events E such that $E \models P$ if, and only if, $e \models P$ for every $e \in E$.

We assume a fixed *set of predicates* $\mathbf{P}$. Further, we assume that there is a basic set of predicates $\mathbf{P}_{\text{basic}} \subseteq \mathbf{P}$ (e.g., $\text{temp} \leq 20^\circ$) and $\mathbf{P}$ is the closure of $\mathbf{P}_{\text{basic}}$ under intersection and negation (i.e., $P_1 \cap P_2 \in \mathbf{P}$ and $\mathbf{E} \setminus P \in \mathbf{P}$ for every $P, P_1, P_2 \in \mathbf{P}$) where $\mathbf{E}$ is a predicate in $\mathbf{P}$, that we usually denote by true . For every $P \in \mathbf{P}$, we define a *size of a predicate* $|P|$ as follows: $|P| = 1$ if $P \in \mathbf{P}_{\text{basic}}$ (i.e., constant size), $|P| = |P_1| + |P_2|$ if $P = P_1 \cap P_2$, and $|P| = |P'| + 1$ if $P = \mathbf{E} \setminus P'$. For every $P \in \mathbf{P}$, we assume that the time to check if $e \models P$ is in $\mathcal{O}(|e|)$.

Complex event logic. In this work, we use the *Complex Event Logic* (CEL) introduced in [22] and implemented in CORE [11] as our basic query language for CER. The syntax of a CEL formula φ is given by the grammar:

$\varphi := R \mid \varphi \text{ AS } X \mid \varphi \text{ FILTER } X[P] \mid \varphi \text{ OR } \varphi \mid \varphi \text{ AND } \varphi \mid \varphi ; \varphi \mid \varphi : \varphi \mid \varphi + \mid \varphi \oplus \mid \pi_L(\varphi)$

where $R \in \mathbf{T}$ is an event type, $X \in \mathbf{X}$ is a variable, $P \in \mathbf{P}$ is a predicate, and $L \subseteq \mathbf{X}$ is a set of variables. Throughout the paper, we use $\varphi \text{ FILTER } (X[P] \wedge X'[P'])$ as syntactic sugar for $(\varphi \text{ FILTER } X[P]) \text{ FILTER } X'[P']$. We define the semantics of a CEL formula φ over a stream $\bar{S}$, recursively, as a set of complex events over $\bar{S}$. In Figure 2, we define the semantics of each CEL operator like in [11, 22].

Example 2.3. Coming back to Examples 2.1 and 2.2, suppose we want to detect a temperature event X above or equal 40° followed by a humidity event Y of less than 25%, which represents a fire with high probability. In addition, we want to check that there is some T -event e between X

$$\begin{aligned}
\llbracket R \rrbracket(\bar{S}) &= \{(i, i, \mu) \mid \text{type}(e_i) = R \wedge \mu(R) = \{i\} \wedge \forall X \neq R. \mu(X) = \emptyset\} \\
\llbracket \varphi \text{ AS } X \rrbracket(\bar{S}) &= \{C \mid \exists C' \in \llbracket \varphi \rrbracket(\bar{S}). \text{interval}(C) = \text{interval}(C') \wedge C(X) = \bigcup_Y C'(Y) \\
&\quad \wedge \forall Z \neq X. C(Z) = C'(Z)\} \\
\llbracket \varphi \text{ FILTER } X[P] \rrbracket(\bar{S}) &= \{C \mid C \in \llbracket \varphi \rrbracket(\bar{S}) \wedge C(X) \models P\} \\
\llbracket \varphi_1 \text{ OR } \varphi_2 \rrbracket(\bar{S}) &= \llbracket \varphi_1 \rrbracket(\bar{S}) \cup \llbracket \varphi_2 \rrbracket(\bar{S}) \\
\llbracket \varphi_1 \text{ AND } \varphi_2 \rrbracket(\bar{S}) &= \llbracket \varphi_1 \rrbracket(\bar{S}) \cap \llbracket \varphi_2 \rrbracket(\bar{S}) \\
\llbracket \varphi_1 ; \varphi_2 \rrbracket(\bar{S}) &= \{C_1 \cup C_2 \mid C_1 \in \llbracket \varphi_1 \rrbracket(\bar{S}) \wedge C_2 \in \llbracket \varphi_2 \rrbracket(\bar{S}) \wedge \text{end}(C_1) < \text{start}(C_2)\} \\
\llbracket \varphi_1 : \varphi_2 \rrbracket(\bar{S}) &= \{C_1 \cup C_2 \mid C_1 \in \llbracket \varphi_1 \rrbracket(\bar{S}) \wedge C_2 \in \llbracket \varphi_2 \rrbracket(\bar{S}) \wedge \text{end}(C_1) + 1 = \text{start}(C_2)\} \\
\llbracket \varphi + \rrbracket(\bar{S}) &= \llbracket \varphi \rrbracket(\bar{S}) \cup \llbracket \varphi ; \varphi + \rrbracket(\bar{S}) \\
\llbracket \varphi \oplus \rrbracket(\bar{S}) &= \llbracket \varphi \rrbracket(\bar{S}) \cup \llbracket \varphi : \varphi \oplus \rrbracket(\bar{S}) \\
\llbracket \pi_L(\varphi) \rrbracket(\bar{S}) &= \{\pi_L(C) \mid C \in \llbracket \varphi \rrbracket(\bar{S})\}
\end{aligned}$$

Fig. 2. The semantics of CEL formulas defined over a stream $\bar{S} = e_1 e_2 \dots e_n$ where each e_i is an event.

and Y with a temperature above or equal 40° that confirms the high temperature (e.g., temperature measures are faulty in the sensor network). The following CEL formula defines this complex event:

$$\varphi_1 := \pi_{X,Y}((T \text{ AS } X; T; H \text{ AS } Y) \text{ FILTER } (T[\text{temp} \geq 40^\circ] \wedge H[\text{hum} < 25\%]))$$

Note that we use variables X and Y to mark the first temperature and the last humidity, respectively, removing the middle event T from the output. In other words, we only want to check the existence of such T -event but do not want it in the final output. One can check that $C_1, C_2 \in \llbracket \varphi_1 \rrbracket(\bar{S}_0)$ among others complex events.

3 Complex event logic under time constraints

In this section, we introduced *timed CEL*, an extension of CEL with time operators for modeling time constraints in complex event recognition (CER). We show how to extend the setting and CEL for modeling these constraints in practice. Further, we show through examples that these operators include most operators introduced in previous works.

Timed complex event logic. We first need to extend each event in a stream with a timestamp to include time constraints. Intuitively, the timestamp of an event models how time passes in the systems when events arrive continuously. These timestamps could represent when the event happened, when the event was measured, or when the event arrived in the system. In any possible scenario, we assume that these timestamps strictly increase with the arrival of events.²

Formally, recall that $\mathbf{T}$ is a fixed set of event types, $\mathbf{A}$ is a fixed set of attributes names, and $\mathbf{D}$ is a fixed set of data values for our events $e : \mathbf{A} \rightarrow \mathbf{D}$ with types in $\mathbf{T}$. A *timed stream* is an (arbitrary long sequence) of pairs:

$$\bar{S} = (e_1, t_1)(e_2, t_2) \dots (e_n, t_n)$$

where e_i is an event and $t_i \in \mathbb{Q}_{\geq 0}$ is its timestamp for every $i \in \mathbb{N}$. We will usually call a pair (e_i, t_i) a *timed event* where e_i is its event and t_i is its timestamp. Further, we assume that timestamps are strictly increasing, namely, $t_1 < t_2 < \dots < t_n$. The stream $\bar{S}_0$ of Figure 1 is also a timed stream where the last line displayed the timestamp of each event (in seconds).

Note that, although we could have modeled timestamps with a special attribute “ts” such that $e(\text{ts}) = t_i$, we preferred to model them as an additional component (i.e., as (e_i, t_i)). The reason is

²In practice, this assumption is not always valid since two events could arrive with the same timestamp or not necessarily in increasing order. Although these are relevant scenarios, we leave them for future work.

$$\begin{aligned}
\llbracket \langle \varphi \rangle_I \rrbracket(\bar{S}) &= \{C \mid C \in \llbracket \varphi \rrbracket(\bar{S}) \wedge t_{\text{end}}(C) - t_{\text{start}}(C) \in I \} \\
\llbracket \varphi_1 ;_I \varphi_2 \rrbracket(\bar{S}) &= \{C_1 \cup C_2 \mid C_1 \in \llbracket \varphi_1 \rrbracket(\bar{S}) \wedge C_2 \in \llbracket \varphi_2 \rrbracket(\bar{S}) \\
&\quad \wedge \text{end}(C_1) < \text{start}(C_2) \wedge t_{\text{start}}(C_2) - t_{\text{end}}(C_1) \in I\} \\
\llbracket \varphi_1 :_I \varphi_2 \rrbracket(\bar{S}) &= \{C_1 \cup C_2 \mid C_1 \in \llbracket \varphi_1 \rrbracket(\bar{S}) \wedge C_2 \in \llbracket \varphi_2 \rrbracket(\bar{S}) \\
&\quad \wedge \text{end}(C_1) + 1 = \text{start}(C_2) \wedge t_{\text{start}}(C_2) - t_{\text{end}}(C_1) \in I\} \\
\llbracket \varphi +_I \rrbracket(\bar{S}) &= \llbracket \varphi \rrbracket(\bar{S}) \cup \llbracket \varphi ;_I (\varphi +_I) \rrbracket(\bar{S}) \\
\llbracket \varphi \oplus_I \rrbracket(\bar{S}) &= \llbracket \varphi \rrbracket(\bar{S}) \cup \llbracket \varphi :_I (\varphi \oplus_I) \rrbracket(\bar{S})
\end{aligned}$$

Fig. 3. The semantics of time operators of timed CEL for every timed CEL formulas $\varphi, \varphi_1, \varphi_2$, a timed stream $\bar{S} = (e_1, t_1) \dots (e_n, t_n)$, and I is an interval over $\mathbb{Q}_{\geq 0}$. Here, recall that $t_{\text{start}}(C)$ is the timestamp of the event in index $\text{start}(C)$.

that timestamps are first citizens in this framework, and we wanted to clearly distinguish between the data (i.e., the events) and the time, which is constantly increasing.

We define *timed Complex Event Logic* (timed CEL for short) as an extension of CEL with additional operators for managing timestamps in time stream. Specifically, a timed CEL formula φ follows the same grammar as a CEL formula where we additionally add the following syntax rules:

$$\begin{array}{ll}
\varphi & ::= \langle \varphi \rangle_I & \text{(time window)} \\
& \mid \varphi ;_I \varphi & \text{(time non-contiguous sequencing)} \\
& \mid \varphi :_I \varphi & \text{(time contiguous sequencing)} \\
& \mid \varphi +_I & \text{(time non-contiguous iteration)} \\
& \mid \varphi \oplus_I & \text{(time contiguous iteration)}
\end{array}$$

where I is an interval over $\mathbb{Q}_{\geq 0}$. We call the above operators the *time operators* of timed CEL, whereas we call the other standard operators just *CEL operators*. The previous syntax extends sequencing and iteration operators in CEL (both in their contiguous and non-contiguous versions) with a corresponding time-variant, where we additionally check a time constraint between complex events. Instead, $\langle \varphi \rangle_I$ is a natural extension of time windows in CER [14, 19].

Timed CEL is an extension of CEL and, as such, we can naturally extend the semantics of CEL operators from streams to timed streams where we just omit the timestamp of timed events (see Figure 2 in Section 2). For the time operators of timed CEL, we define their semantics recursively in Figure 3. In the sequel, we will use the notation $\sim c$ in timed CEL formulas to specify an interval $I = \{p \in \mathbb{Q}_{\geq 0} \mid p \sim c\}$ over $\mathbb{Q}_{\geq 0}$ for any $c \in \mathbb{Q}_{\geq 0}$ and $\sim \in \{\leq, \geq, <, >, =\}$. For example, we write $\langle A ;_{>2} B \rangle_{\leq 5}$ to denote the formula $\langle A ;_{(2,\infty)} B \rangle_{[0,5]}$. Next, we show some examples on how to use and compose the time operators of timed CEL to define relevant patterns over timed streams.

Example 3.1 (Continued Example 2.3). For finding relevant complex events in a fire scenario, we also want to restrict that the time difference between the high temperature X and low humidity Y is less than five seconds. Further, we expect that the confirmation of the high temperature (i.e., the middle event T) occurs in less than one second from X . We can define these time restrictions in timed CEL as follows:

$$\varphi'_1 := \pi_{X,Y}(\langle T \text{ AS } X ;_{\leq 1} T ; H \text{ AS } Y \rangle_{\leq 5} \text{ FILTER } (T[\text{temp} \geq 40^\circ] \wedge H[\text{hum} < 25\%]))$$

We can check that now $C_1 \in \llbracket \varphi'_1 \rrbracket(\bar{S}_0)$ but $C_2 \notin \llbracket \varphi'_1 \rrbracket(\bar{S}_0)$.

Example 3.2. As an example of using time iteration, suppose that we want to see how temperature changes when humidity increases from less than 30% to over 30% [22, Ex.3]. Further, the time

difference between consecutive events must be less than one second. We can define this query as:

$$\varphi_2 := \pi_{X,Y,T} ((H \text{ AS } X :_{\leq 1} (T \oplus_{\leq 1}) :_{\leq 1} H \text{ AS } Y) \text{ FILTER } (X[\text{hum} < 30\%] \wedge Y[\text{hum} > 30\%]))$$

Note that we use the contiguous sequencing and iteration to capture all consecutive events between X and Y . Furthermore, time iteration is useful to impose a repetitive time restriction over a sequence of contiguous events. One can check that:

$$(4, 8, \{X \mapsto \{4\}, Y \mapsto \{8\}, T \mapsto \{5, 6, 7\}, H \mapsto \emptyset\}) \in \llbracket \varphi_2 \rrbracket (\bar{S}_0).$$

Timed CEL in practice. Arguably, the operators introduced in timed CEL cover most of the use of time restrictions in CER. Below, we discuss how timed CEL can model previous proposals, dividing them into three groups. Our goal is not to be exhaustive but to show how different flavors of timed constraints appear in CER and coincide with timed CEL.

Time windows. Most previous CER query languages introduce time restriction as a form of a sliding window [19, 37], declared by a WITHIN clause. For example, consider the SASE+ query in [1]:

```
PATTERN SEQ(Shelf a, ~(Register b), Exit c)
WHERE skip_till_next_match(a, b, c) { [tag_id] }
WITHIN 12 hours
```

Here, the WITHIN clause forces that the pattern must appear in a time window of the last 12 hours. Similarly, several systems use the WITHIN clause (see, e.g., [11, 13, 29, 31]). Specifically, if a user wants to find the pattern φ between c hours (or any time scale) for some value c , we can express it in timed CEL as $\langle \varphi \rangle_{\leq c}$.

Notice that we limit the above discussion to use time-based windows, contrary to count-based windows where the WITHIN clause specifies how many events must be inside the windows (e.g., WITHIN 100 events). For count-based windows, one can easily use the AND operator to declare that the matched substream must have some specific length. Also, we compared timed CEL with operators in CER for imposing time restrictions over a pattern. Specifically, CER systems and, in general, stream data management systems also use window operators to restrict the amount of data where it must evaluate the query, which we call here a *data window*. For example, *tumbling windows* is a data window that splits the stream into consecutive sequences of equal size (i.e., the same number of events), and the query is evaluated over each sequence. We refer the reader to [37] that surveys several data windows types used in stream processing systems. Of course, timed CEL cannot necessarily express data windows, given that the time operators are designed to impose restrictions related to time and not associated with data.

Time sequencing and time iteration. These operators are less common in the literature. One example of a system that uses time sequencing is Padres [26] which uses $;_I$ for checking a time constraint with two relevant events (not necessarily contiguous). As an illustration, consider the following (simplified) example from [14]:

$$(B(Y=10)) ;_{[\text{timespan}:5]} C(Z<5))_{[\text{within}:15]}$$

Intuitively, this query in Padres means that the user wants to find an event of type B with attribute $Y=10$, followed by an event of type C with attribute $Z<5$. Further, the operator $;_{[\text{timespan}:5]}$ restricts that the timespan between B and C has to be greater than 5 seconds, and $(\dots)_{[\text{within}:15]}$ to be lower than 15 seconds. This query is equivalent to $\langle (B \text{ FILTER } B[Y=10]) ;_{\geq 5} (C \text{ FILTER } C[Z<5]) \rangle_{\leq 15}$.

To the best of our knowledge, we are not aware of CER systems that use time iteration like $+_I$ or $\oplus_I$. Probably, the reason behind this limitation is that CER systems struggle with iteration (i.e., without time) and, usually, they provide a limited use of this operator [1, 15]. One can see time iteration as the natural generalization of time sequencing, and for this reason, it is natural to add them as a primary operator of timed CEL.

Combining all time operators. We are not aware of a CER system or CER query language with the same expressibility as timed CEL. Still, there are query languages that combine time windows and time sequencing. One example is Padres [26], which we previously discussed above. Another more recent proposal is *subsequence-queries with wildcards and gap-size constraints* (swg-queries for short) introduced in [24]. A swg-query Q is a triple (s, w, c) where $s = P_1 \dots P_k$ is a sequence of predicates (called *types* in [24]), $w \in \mathbb{Q}_{\geq 0} \cup \{\infty\}$ is a global window size, and $c = (I_1, \dots, I_{k-1})$ is a tuple of local gap-size constraints where each I_i is an interval over $\mathbb{Q}_{\geq 0}$ for every $i \in [k-1]$. A time stream $\bar{S} = (e_1, t_1)(e_2, t_2) \dots (e_n, t_n)$ satisfies Q if we can find a subsequence $(e'_1, t'_1) \dots (e'_k, t'_k)$ of $\bar{S}$ such that $e'_i \models P_i$, the total time is less than w (i.e., $t_k - t_1 \leq w$), and each consecutive pair of timed events (e'_i, t'_i) and (e'_{i+1}, t'_{i+1}) satisfies the local gap-size constraint I_i (i.e., $t'_{i+1} - t'_i \in I_i$). The matches of Q over $\bar{S}$ are all subsequences $(e'_1, t'_1) \dots (e'_k, t'_k)$ that witness that $\bar{S}$ satisfies Q .

We must note that our definition of swg-queries slightly differs from [24] to fit our purpose better. First, [24] uses window size and gap-sizes in $\mathbb{N} \cup \{\infty\}$, since they model time discretely. Instead, we present above the natural extension of swg-queries from $\mathbb{N}$ to $\mathbb{Q}_{\geq 0}$. Second, [24] considers variables (called wildcards) in the sequence $P_1 \dots P_k$ where each variable repetition must match the same type of event. This feature corresponds to some form of a join operation (called correlation in CER), a feature that we do not include in this work for the sake of simplification (see Section 7).

One can easily see that any swg query $Q = (s, w, c)$ like above can be represented in timed CEL as follows. Let $\mathbf{T} = \{R_1, \dots, R_m\}$ be the set of all event types, and define the CEL formula $\varphi_{\mathbf{T}} := R_1 \text{ OR } R_2 \text{ OR } \dots \text{ OR } R_m$ that its true for every event. Then, one can easily check that the following timed CEL formula φ_Q is equivalent to Q :

$$\varphi_Q := \langle (\varphi_{\mathbf{T}} \text{ AS } X_1) \text{ FILTER } X_1[P_1] ;_{I_1} \dots ;_{I_{k-1}} (\varphi_{\mathbf{T}} \text{ AS } X_k) \text{ FILTER } X_k[P_k] \rangle_{\leq w} \quad (*)$$

Therefore, one can consider swg-queries as a (strict) subfragment of timed CEL. For this reason, in the following we call a timed CEL formula φ a *swg query* if it has the form of (*).

4 Timed complex event automata

To understand the expressiveness of timed CEL and design efficient algorithms, we introduce here its automata counterpart, which we call timed Complex Event Automata (timed CEA). This model is the natural combination of complex event automata [11, 22] (CEA) and timed automata, namely, we extend CEA with clocks.

In the following, we introduce the model of timed CEA and study its expressive power with timed CEL. Interestingly, we show that both models are equally expressive, providing evidence of the robustness of timed CEL as a logical language for CER with time constraints. We start by first recalling the notion of CEA to introduce later timed CEA.

Complex Event Automata. A *Complex Event Automata* (CEA) is a tuple $\mathcal{A} = (Q, \mathbf{P}, \mathbf{X}, \Delta, q_0, F)$ where Q is a finite set of states, $\mathbf{P}$ is the set of predicates, $\mathbf{X}$ is a finite set of variables, $\Delta \subseteq Q \times \mathbf{P} \times 2^{\mathbf{X}} \times Q$ is a finite relation (called the transition relation), $q_0 \in Q$ is the initial state, and F is the set of final states. A *run* ρ of $\mathcal{A}$ over the stream $\bar{S} = e_1 e_2 \dots e_n$ from position i to j is a sequence:

$$\rho := p_i \xrightarrow{P_i, L_i} p_{i+1} \xrightarrow{P_{i+1}, L_{i+1}} p_{i+2} \xrightarrow{P_{i+2}, L_{i+2}} \dots \xrightarrow{P_j, L_j} p_{j+1}$$

where $p_i = q_0$, $(p_k, P_k, L_k, p_{k+1}) \in \Delta$, and $e_k \models P_k$ for all $k \in [i..j]$. We say that the run is *accepting* if $p_{j+1} \in F$. A run ρ from positions i to j like above defines the complex event $C_\rho = (i, j, \mu_\rho)$ such that $\mu_\rho(X) = \{k \in [i..j] \mid X \in L_k\}$ for every $X \in \mathbf{X}$. Note that the starting and ending positions i, j of the run define the interval of the complex event, and the labels $L_k \subseteq \mathbf{X}$ define the mapping μ_ρ of C_ρ . We define the set of all complex events of $\mathcal{A}$ over $\bar{S}$ as:

$$\llbracket \mathcal{A} \rrbracket(\bar{S}) = \{C_\rho \mid \rho \text{ is an accepting run of } \mathcal{A} \text{ over } \bar{S}\}$$

Timed CEA. The model of timed CEA is the natural extension of CEA with *clocks*, similarly as timed automata is the extension of finite automata [3]. First we introduce clock conditions to then introduce the model.

Let Z be a finite set of *clocks*. A *clock condition* over Z is an expression following the grammar:

$$\gamma := \text{true} \mid z = c \mid z < c \mid z \leq c \mid z \geq c \mid z > c \mid \gamma \wedge \gamma \mid \gamma \vee \gamma$$

where $z \in Z$ and $c \in \mathbb{Q}_{\geq 0}$. We define the size $|\gamma|$ of a clock condition γ as the number of operations in γ . We denote by C_Z the set of *all clock conditions* over Z . A *clock valuation* $v : Z \rightarrow \mathbb{R}_{\geq 0}$ is a partial mapping that assigns to a clock $z \in Z$ a timestamp $v(z)$. If $z \notin \text{dom}(v)$, it represents that z *has not been initialized*. Given a clock condition γ and a clock valuation v , we say that v *satisfies* γ , denoted by $v \models \gamma$, if all variables in γ are in $\text{dom}(v)$ and the expression resulting by replacing every clock z by $v(z)$ in γ evaluates to true. For example, the clock valuation $v := \{x \mapsto 5, y \mapsto \frac{1}{2}\}$ satisfies $\gamma_1 := x \geq 4 \vee y < \frac{1}{2}$ but does not satisfy $\gamma_2 := z \leq \frac{5}{3} \wedge y \geq 1$ (i.e., $z \notin \text{dom}(v)$).

The following operations on clock valuations will simplify the presentation of timed CEA. Let v be a clock valuation. For $t \in \mathbb{Q}_{\geq 0}$, we define the *update of v by t* as the clock valuation $v + t$ such that, for every $z \in Z$, $[v + t](z) = v(z) + t$ whenever $z \in \text{dom}(v)$, and undefined, otherwise. Furthermore, for a set $Z \subseteq Z$ of clocks we define the *reset of v on Z* as the clock valuation $\text{reset}_Z(v)$ such that $\text{dom}(\text{reset}_Z(v)) = \text{dom}(v) \cup Z$ where $[\text{reset}_Z(v)](z) = 0$ when $z \in Z$ and $[\text{reset}_Z(v)](z) = v(z)$ when $z \in \text{dom}(v) \setminus Z$.

A *timed Complex Event Automata* (timed CEA for short) is a tuple:

$$\mathcal{T} = (Q, P, X, Z, \Delta, q_0, F)$$

where Q, P, X, q_0 , and F are exactly as for CEA. In addition, Z is the set of clocks and $\Delta \subseteq Q \times P \times C_Z \times 2^X \times 2^Z \times Q$ is the finite transition relation. The two new components γ and Z in a transition $\tau = (p, P, \gamma, L, Z, q)$ represent the condition on the clocks that the automata must satisfy before firing τ and the set of clocks that are reset after firing τ , respectively. We define the size $|\mathcal{T}|$ of $\mathcal{T}$ as the sum of the number of states and transitions, namely, $|\mathcal{T}| = |Q| + \sum_{(p, P, \gamma, L, Z, q)} |\gamma| + |P| + |L| + |Z|$.

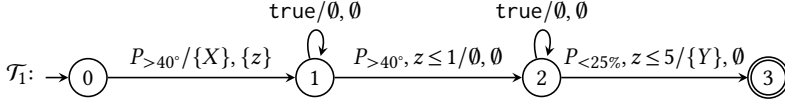
Let $\bar{S} = (e_1, t_1) \dots (e_n, t_n)$ be a timed stream. A configuration of $\mathcal{T}$ over $\bar{S}$ is a pair (q, v) where $q \in Q$ is the current state of $\mathcal{T}$ and v is a clock valuation representing the current values of the clocks in $\text{dom}(v)$. We define the *time difference at position $i \in [n]$* as $\delta t_i = t_i - t_{i-1}$ for every $i \in [n]$ where $\delta t_1 = t_1$ when $i = 1$. Then, a *run ρ of $\mathcal{T}$ over $\bar{S}$ from position i to j* is a sequence:

$$\rho := (p_i, v_i) \xrightarrow{P_i, \gamma_i / L_i, Z_i} (p_{i+1}, v_{i+1}) \xrightarrow{P_{i+1}, \gamma_{i+1} / L_{i+1}, Z_{i+1}} \dots \xrightarrow{P_j, \gamma_j / L_j, Z_j} (p_{j+1}, v_{j+1})$$

where $p_i = q_0$, v_i is the *trivial* clock valuation (i.e., $\text{dom}(v_i) = \emptyset$), and, for every $k \in [i..j]$, the following four conditions are satisfied: (1) $(p_k, P_k, \gamma_k, L_k, Z_k, p_{k+1}) \in \Delta$, (2) $e_k \models P_k$, (3) $v_k + \delta t_k \models \gamma_k$, and (4) $v_{k+1} = \text{reset}_{Z_k}(v_k + \delta t_k)$. Conditions (1) and (2) are verbatim from CEA, (3) checks that the clock values must satisfy the clock conditions at each position, and (4) specifies that clocks in Z_k are reset after updating them with the time difference at position k . We say that ρ like above is an *accepting run* if $p_{j+1} \in F$. Similar to CEA, we define the complex event $C_\rho = (i, j, \mu_\rho)$ of ρ where $\mu_\rho(X) = \{k \in [i..j] \mid X \in L_k\}$ for every $X \in X$, and the *output set* of $\mathcal{T}$ over $\bar{S}$ as $\llbracket \mathcal{T} \rrbracket(\bar{S}) = \{C_\rho \mid \rho \text{ is an accepting run of } \mathcal{T} \text{ over } \bar{S}\}$.

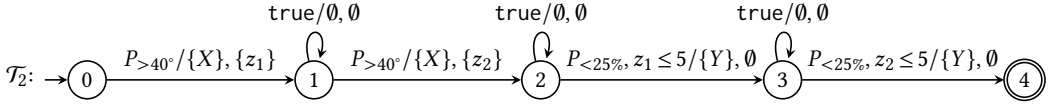
Note that a run of a timed CEA must start with all clocks empty (i.e., clocks are not initialized by default). Then, to use a clock z , the automata first has to reset z in a run. This technical detail is different from timed automata [3], and it is necessary to be aligned with the principles behind the semantics of CEL. If not, the automata could have the power to check a property with the event just before the interval $[i..j]$ (i.e., e_{i-1}), and thus, the complex event C_ρ will not depend only on $[i..j]$.

Example 4.1. The following timed CEA uses a single clock z and set of variables $X = \{X, Y\}$:



where $P_{>40^\circ} := temp > 40^\circ$, $P_{<25\%} := hum < 25\%$, and true denotes the predicate E. One can check that $\mathcal{T}_1$ defines the same query as formula φ_1 from Example 3.1.

Example 4.2. To illustrate the advantages of more clocks, suppose that to reinforce formula φ_1 (see Example 2.3), we want to detect two pairs (X, Y) (i.e., high temperature / low humidity) where each pair must be between less than five seconds. Further, to confirm the high temperatures and low humidities, we expect to see the temperatures first and then the humidities. For defining this query with a timed CEA, we need states to check the pattern $T; H; H$ and two clocks z_1 and z_2 to check the time differences for each pair (X, Y) , respectively. The timed CEA $\mathcal{T}_2$ specifies this query:



The expressive power of timed CEL. We proceed by measuring the expressiveness of timed CEL with its automata counterpart, timed CEA. We start by showing that any timed CEL can be compiled into a timed CEA that defines the same query as expected.

PROPOSITION 4.3. *For every timed CEL formula φ there exists a timed CEA $\mathcal{T}_\varphi$ such that $\llbracket \varphi \rrbracket(\bar{S}) = \llbracket \mathcal{T}_\varphi \rrbracket(\bar{S})$ for every stream $\bar{S}$.*

PROOF SKETCH. The proof goes by induction over the formula, showing how to compile each operator into a timed CEA. The standard CEL operators follow a similar construction like in [22], but this time one also needs to take care of clocks, keeping disjoint sets of clocks for each subautomaton. For the new timed operators, one can use a new clock for each operator, for checking the condition of the time window, time sequencing, or time iteration. One can find the full proof in the online version [18]. $\square$

For the other direction, it is less clear whether any timed CEA can be defined by some timed CEL formula. Indeed, timed CEA can check overlapping timed restrictions (like in Example 4.2) while timed CEL can only check nested time constraints with the time window operator (i.e., $\langle \cdot \rangle_I$) or immediate time constraints with the timed sequencing (i.e., $:_I$ or $:_I$) or timed iteration (i.e., $+_I$ or $\oplus_I$). Interestingly, timed CEL formulas can simulate overlapping time constraints by using variables (AS), projection (π_L), and conjunction (AND). For instance, we can define the query of Example 4.2 with the following timed CEL query:

$$\varphi_3 := \pi_{X,Y} \left(\left[\langle T \text{ AS } X; T \text{ AS } X; H \text{ AS } Y \rangle_{\leq 5}; H \text{ AS } Y \text{ AND } T \text{ AS } X; \langle T \text{ AS } X; H \text{ AS } Y; H \text{ AS } Y \rangle_{\leq 5} \right] \text{ FILTER } (X[temp \geq 40^\circ] \wedge Y[hum < 25\%]) \right)$$

As the following result shows, we can follow a similar strategy for every timed CEA.

PROPOSITION 4.4. *For every timed CEA $\mathcal{T}$ there exists a timed CEL formula $\varphi_{\mathcal{T}}$ such that $\llbracket \mathcal{T} \rrbracket(\bar{S}) = \llbracket \varphi_{\mathcal{T}} \rrbracket(\bar{S})$ for every stream $\bar{S}$. Furthermore, $\varphi_{\mathcal{T}}$ does not use any operator in $\{;, +, :_I, :_I, +_I, \oplus_I\}$.*

PROOF SKETCH. The proof starts by reducing the construction from a timed CEA $\mathcal{T}$ with multiple clocks to a timed CEA with a single clock $\mathcal{T}_1$. For each clock z , one can construct a timed CEA $\mathcal{T}_z$ that simulates $\mathcal{T}$, but it only verifies the clock z and marks with temporal variables all the transitions that are used during a run. Then, for all the clocks $z_1, \dots, z_k$ used by $\mathcal{T}$, we can take

the conjunction of $\mathcal{T}_{z_1}, \dots, \mathcal{T}_{z_k}$ and then project out the temporal variables. Therefore, the problem reduces to encode with timed CEL the behavior of a timed CEA $\mathcal{T}_1$ with a single clock.

For encoding a timed CEA $\mathcal{T}_1$ with a single clock z , we use two automata constructions to reduce the complexity of how the clock z is used. For the first construction, we show that $\mathcal{T}_1$ can be reduced to a timed automaton $\mathcal{T}'_1$ such that z is checked at most k -times after it is reset, where k is a fixed constant. $\mathcal{T}'_1$ is called a *k-check bounded timed CEA*. This reduction helps to limit the number of times a clock is used after a reset, similar to a timed CEL formula where each clock is used exactly once (i.e., after a reset). For the second construction, we show that the *k-check bounded automaton* $\mathcal{T}'_1$ can be specified as the conjunction of two *k-check bounded automaton* $\mathcal{T}''_1$ and $\mathcal{T}'''_1$ such that each automaton can either reset a clock or check a clock during a transition, but not both at the same time. We call this condition a *simplified timed CEA*.

Finally, we prove that any simplified and *k-check bounded timed CEA* with a single clock can be defined by a timed CEL formula. This construction follows a similar strategy to the Kleene Theorem (for regular expressions), where the formula is constructed by induction on the set of states and the number of checks that are used during a run. More details of the proof in the online version [18]. $\square$

By combining Proposition 4.3 and 4.4, we can show that both formalism are equally expressive.

THEOREM 4.5. *Timed CEL and timed CEA are equally expressive, namely, one can be defined by the other and vice versa.*

It is important to note that an analog result was proven in [5], showing that timed automata are equally expressive than Timed Regular Expressions (TREs). TREs are an extension of regular expressions with time windows, renaming, conjunction, and the so-called absorbing concatenation and absorbing iteration. The paper aimed to extend the expressive power of regular expressions to capture timed automata. Although Proposition 4.4 is inspired by [5], arguably Theorem 4.5 is novel, both in its statement and proof, for the following reasons. First, for reaching Proposition 4.4 we only have to extend CEL with time windows, an operator already presented in practical proposals in CER. In particular, CEL already included all other operators needed for the characterization. In contrast, TREs needed several new operators for regular expressions to capture the expressive power of time automata. Second, although one can make a parallel between the operators between timed CEL and TREs, they are different in syntax and semantics. Indeed, timed CEL includes outputs (i.e., complex events) and operators like projection and filtering, which are features that are not present in TREs. For this reason, the proof of Proposition 4.4 is not a direct consequence of the results in [5]. Finally, the proof of Proposition 4.4 differs from the one in [5]. The proof strategy of reducing the problem to single-clock automata is similar; however, the proof differs from there. Indeed, we cannot follow the same approach as in [5] given the semantics of timed CEL, namely, by using the so-called *absorbing* concatenation and iteration and constructing quasilinear equations. We cannot simulate these operators in timed CEL, and therefore, we have to construct the formula for a single-clock timed CEA directly, as we explained in the proof sketch of Proposition 4.4.

5 On the determinization of timed complex event automata

Now that we understand the expressive power of timed CEL concerning timed CEA, we move next to the efficient evaluation of timed CEL or, more concretely, of timed CEA. A critical aspect for efficient query evaluation is whether timed CEA are determinizable or not. We say that a timed CEA $\mathcal{T} = (Q, P, X, Z, \Delta, q_0, F)$ is *deterministic* if for every pair of transitions $\tau_1 = (p, P_1, \gamma_1, L_1, Z_1, q_1)$ and $\tau_2 = (p, P_2, \gamma_2, L_2, Z_2, q_2)$ in Δ , if $P_1 \cap P_2 \neq \emptyset$ and $\gamma_1 \wedge \gamma_2$ is satisfiable (i.e., there exists v such

that $v \models \gamma_1$ and $v \models \gamma_2$), then $L_1 \neq L_2$. In other words, for any configuration (p, v) of $\mathcal{T}$, an event e can trigger transitions τ_1 and τ_2 simultaneously only if they produce different outputs.

Example 5.1. $\mathcal{T}_2$ in Example 4.2 is a deterministic timed CEA, but $\mathcal{T}_1$ in Example 4.1 is not since the predicates of the outgoing transitions of state ① intersect and produce the same output.

The importance of determinism for a timed CEA $\mathcal{T}$ is that for every timed stream $\bar{S}$ and complex event $C \in \llbracket \mathcal{T} \rrbracket(\bar{S})$, there exists a unique run ρ of $\mathcal{T}$ over $\bar{S}$ that produces C , namely, $C = C_\rho$. This implies that there exists a correspondence between runs and outputs, which is very useful for designing efficient streaming (and non-streaming) enumeration algorithms. Indeed, most of the algorithms in the query evaluation literature assumed that the automata model that represents a query is deterministic (e.g., [7, 17, 32]). Therefore, being deterministic is desirable for the efficient evaluation of timed CEA and the enumeration of their outputs. Nevertheless, it is important to say that the size of the determinized automaton is exponential. However, if we think about *data complexity*, the blow-up of the automata is in the size of the query (which is small) and not in the size of the stream (which is big).

Unfortunately, not all timed CEA admit an equivalent deterministic one. Specifically, we say that a timed CEA $\mathcal{T}$ is *determinizable* if there exists a deterministic timed CEA $\mathcal{T}'$ equivalent to $\mathcal{T}$ (i.e., $\llbracket \mathcal{T} \rrbracket(\bar{S}) = \llbracket \mathcal{T}' \rrbracket(\bar{S})$). This issue of non-determinizable timed CEA already holds for timed automata, where it is well-known that they are not always determinizable [3, 4, 8]. Moreover, determining if a timed automaton is determinizable is an undecidable problem [16], and that property is easy to extend to timed CEA. As an example, for the following timed CEL formula, there exists no deterministic timed CEA that defines it:

$$\varphi_{\text{non-det}} := \pi_0(A; A;_{=1} A).$$

Note that the above formula is analog to the non-determinizable timed automata in [3, Figure 12]. Intuitively, $\varphi_{\text{non-det}}$ looks for complex events (i, j, μ_0) of three A -events e_i , e_k , and e_j with $k \in [i..j]$ such that the time difference between e_k and e_j is exactly 1. A deterministic timed CEA must keep track of the timestamp of all A -events after e_i to verify that such e_k exists when the last event e_j arrives. Thus, we will need an unbounded number of clocks to define $\varphi_{\text{non-det}}$.

Although timed CEA are not always determinizable, we strive to find a relevant class of timed CEA where each automaton is determinizable. Formally, we say that a class C of timed CEA is *closed under determinization* if, and only if, every $\mathcal{T} \in C$ is determinizable. In the following, we present and study a class of timed CEA that is closed under determinization. Furthermore, we show that this class already contained an interesting class of timed CEL formulas that admit a representation as a deterministic timed CEA.

Synchronous resets timed CEA. A timed CEA $\mathcal{T} = (Q, P, X, Z, \Delta, q_0, F)$ has *synchronous resets* if for every $\bar{S} = (e_1, t_1) \dots (e_n, t_n)$ and for every two runs of $\mathcal{T}$ over $\bar{S}$ from position i to j :

$$(p_i, v_i) \xrightarrow{P_i, \gamma_i / L_i, Z_i} \dots \xrightarrow{P_j, \gamma_j / L_j, Z_j} (p_{j+1}, v_{j+1}) \quad \text{and} \quad (p'_i, v'_i) \xrightarrow{P'_i, \gamma'_i / L_i, Z'_i} \dots \xrightarrow{P'_j, \gamma'_j / L_j, Z'_j} (p'_{j+1}, v'_{j+1})$$

it holds that $Z_k = Z'_k$ for every $k \in [i..j]$. Note that the two runs must have the same output $L_i, \dots, L_j$, and they are not necessarily accepting runs; thus, the condition must hold for every run (i.e., accepting or not) or extension of them. We define the class of *synchronous resets timed CEA* as all timed CEA that have synchronous resets.

Example 5.2. One can check that, although $\mathcal{T}_1$ is not deterministic (Example 5.1), it has synchronous resets since the clock is only reset at the beginning of the run.

One can easily check that every deterministic timed CEA has synchronous resets but not necessarily the other way around. Interestingly, we can show that the class of synchronous resets timed CEA is closed under determinization and, thus, $\mathcal{T}_1$ is determinizable.

THEOREM 5.3. *For every synchronous resets timed CEA $\mathcal{T} = (Q, P, X, Z, \Delta, q_0, F)$, there exists a deterministic timed CEA $\mathcal{T}'$ such that $\llbracket \mathcal{T} \rrbracket(\bar{S}) = \llbracket \mathcal{T}' \rrbracket(\bar{S})$. Furthermore, $|\mathcal{T}'| \in O(2^{|Q|+2|\Delta|} \cdot |\mathcal{T}|)$.*

PROOF SKETCH. The construction of $\mathcal{T}'$ follows the standard subset construction where 2^Q are the states of $\mathcal{T}'$. One has to additionally solve the use of predicates and time conditions in the transitions. Given that both are closed under conjunction and negation, we can use *types* to represent subsets of them. Finally, given that the resets of competing runs are synchronized, we can reset without considering conflicts between runs. The full proof is in the online version [18]. $\square$

A natural question at this point is whether one can decide if a timed CEA has synchronous resets or not. The next result shows that this property is decidable in PSPACE.

THEOREM 5.4. *The problem of deciding if a timed CEA $\mathcal{T}$ has synchronous resets is PSPACE-complete.*

PROOF SKETCH. For the PSPACE upper bound, we explore non-deterministically all of the different pairs of runs of the timed CEA. We can do so by grouping all of the runs that go through the same clock regions of the automaton, a technique introduced originally in [3]. That way, the number of possible next runs is polynomial. If any two runs do not have the same clock resets, then the timed CEA does not have synchronized resets.

For the PSPACE-hardness, we reduce from the time automaton *emptiness* problem. Given a timed automaton $\mathcal{A}$, we construct a timed CEA $\mathcal{T}_{\mathcal{A}}$ where we mark all of the transitions with different variables so that the resulting timed CEA is deterministic. Then, we add two self-loops resetting different sets of clocks in the final states so that a run in $\mathcal{A}$ reaches a final state if, and only if, two runs reset different sets of clocks in $\mathcal{T}_{\mathcal{A}}$. We present the full proof in [18]. $\square$

The determinization of timed automata is a relevant topic in the area. Indeed, one can translate classes of determinizable timed automata to timed CEA, and vice versa, by reinterpreting the notion of output and predicates. For instance, synchronous resets are more powerful than *event-clock automata* introduced in [4] because every event-clock automata has synchronous resets. Other classes of determinizable timed automata, like timed automata with integer resets [34], non-Zeno timed automata [6], or history-deterministic timed automata [23], are incomparable with synchronous resets. Theorem 5.3 can also be recovered from general determinization procedures (see [8, 27]), by unfolding the timed CEA with synchronous resets into an infinite tree and determinizing it afterwards, because the infinite tree would be $|Z|$ -bounded, where $|Z|$ is the number of clocks. All in all, synchronous resets timed CEA are incomparable to previous proposals in terms of the setting and the use of clocks. Moreover, although the determinization of synchronous resets may be recovered from more general proposals, one could argue that the determinization procedure of synchronous resets is simpler, computable *on-the-fly* (i.e., one can compute the next state in linear time), and one can decide the property in PSPACE (i.e., for [8, 27] it is undecidable). Finally, as we show next, the class of synchronous resets contains a relevant class of timed CEL formulas.

We say that a timed CEL formula ψ is *simple* if ψ does not use projection $\pi_L(\cdot)$ or time window $\langle \cdot \rangle_I$ as a subformula. Further, a timed CEL formula ϕ is *windowed* if it respects the following grammar:

$$\phi := \varphi \mid \psi \mid \phi \text{ AS } X \mid \phi \text{ FILTER } X[P] \mid \phi \text{ OR } \phi \mid \phi \text{ AND } \phi \mid \langle \phi \rangle_I$$

where φ is a standard CEL formula (i.e., no time operators), ψ is a simple timed CEL formula, P is a predicate, and I is an interval. In other words, windowed CEL formulas have two levels. The first level allows either the free use of all timed CEL operators except for projection and time windows

or the use of no time operator at all. The second level allows the use of sets of operators and filters plus time windows. For instance, all swg queries (i.e., queries of the form $(*)$) are windowed. Interestingly, one can compile any windowed timed CEL formula into timed CEA with synchronous resets, and, therefore, they are determinizable.

THEOREM 5.5. *Let ϕ be a windowed timed CEL formula. Then, there exists a two-clock timed CEA with synchronous resets $\mathcal{T}$ such that $\llbracket \mathcal{T} \rrbracket(\bar{S}) = \llbracket \phi \rrbracket(\bar{S})$.*

The previous result pinpointed a timed CEL fragment that generalizes swg queries and single windows queries (e.g., SASE+[1], CORE[11]). Moreover, each formula can be compiled into a deterministic timed CEA. Therefore, windowed timed CEL is an expressive fragment and a good candidate for efficient streaming evaluation. Although this does not mean that windowed timed CEL can be evaluated efficiently, at least one could exploit the determinism to process them.

6 Towards the efficient evaluation of timed complex event automata

In this last section, we will look for efficient algorithms for evaluating (deterministic) timed CEA in the streaming scenario of CER. Here, we will provide a partial answer by showing the first algorithm for evaluating a restricted class of deterministic timed CEA with a single clock and monotonic clock conditions. We show that a streaming enumeration algorithm for this class is already non-trivial and, further, that it already includes an interesting subclass of queries.

The plan for this section is to first present the problem with the algorithmic guarantees we strive for to solve it. Then, we define the class of monotonic timed CEA with the statement of the main algorithmic result. Finally, we provide an overview of the algorithm and some of its consequences. Given space restrictions, we move all the technical details of the algorithm to the online version [18].

The evaluation problem. A timed CEA $\mathcal{T}$ defines a function from a timed stream $\bar{S}$ to a set of complex events $\llbracket \mathcal{T} \rrbracket(\bar{S})$. In practice, the stream is arbitrarily long and, thus, users want results as soon as they occur. Then, it is natural to define the restriction of $\llbracket \mathcal{T} \rrbracket(\bar{S})$ at position $j \in [\lvert \bar{S} \rvert]$ as $\llbracket \mathcal{T} \rrbracket_j(\bar{S}) = \{C \in \llbracket \mathcal{T} \rrbracket(\bar{S}) \mid \text{end}(C) = j\}$, namely, all complex events that end at position j . Given the semantics of timed CEL and timed CEA, if a complex event starts and ends at positions i and j , respectively, then it only depends on the substream between i -th and j -th positions. Therefore, as soon as the j -th timed event arrives, we can safely return all complex events $\llbracket \mathcal{T} \rrbracket_j(\bar{S})$.

Let C be a class of timed CEA. The main evaluation problem that we aim to solve, parameterized by C , can be formalized as follows:

Problem:	TIMEDCEAEVALUATION[C]
Input:	A timed CEA $\mathcal{T} \in C$ and a timed stream $\bar{S}$
Output:	Enumerates all complex events $\llbracket \mathcal{T} \rrbracket_j(\bar{S})$ at each position $j \in [\lvert \bar{S} \rvert]$

Our main goal is to find algorithms for TIMEDCEAEVALUATION[C] that evaluates $\mathcal{T}$ reading $\bar{S} = (e_1, t_1) \dots (e_n, t_n)$ event by event and enumerates $\llbracket \mathcal{T} \rrbracket_j(\bar{S})$ right after reading the j -th events. For this, we assume that one can only access $\bar{S}$ through a method $\text{yield}(\bar{S})$ such that the j -th call to this function retrieves (e_j, t_j) if $j \leq n$, or EOS (End-Of-Stream) otherwise.

A *streaming evaluation algorithm* for TIMEDCEAEVALUATION[C] is an algorithm that receives as input $\mathcal{T} \in C$ and consumes $\bar{S}$ by calling $\text{yield}(\bar{S})$, maintaining a data structure DS such that, after the j -th call to $\text{yield}(\bar{S})$, it receives (e_j, t_j) and operates over DS in two phases. The first phase, called the *update phase*, updates DS with (e_j, t_j) . The second phase, called the *enumeration phase*, enumerates the complex events in $\llbracket \mathcal{T} \rrbracket_j(\bar{S})$ one by one using DS. After the second phase is ready, it proceeds to the next call to $\text{yield}(\bar{S})$, and so on.

We consider two parameters to measure the efficiency of a streaming evaluation algorithm $\mathcal{S}$. First, we say $\mathcal{S}$ has *f-update time* for some function f if the time during the update phase of (e_j, t_j) is at most $O(f(\mathcal{T}, e_j))$. Second, we say that $\mathcal{S}$ has *output-linear delay* if there exists a constant $d \in \mathbb{N}$ such that, for every j , during the enumeration phase of $\llbracket \mathcal{T} \rrbracket_j(\bar{S}) = \{C_1, \dots, C_m\}$, the time taken between retrieving C_k and C_{k+1} (i.e., the *delay*) is bounded by $d \cdot |C_{k+1}|$. This implies that the time to output the first complex event is at most $d \cdot |C_1|$ and constant time after the last output. In particular, $\mathcal{S}$ takes constant time to end the enumeration phase if there is no output (i.e., $\llbracket \mathcal{T} \rrbracket_j(\bar{S}) = \emptyset$).

For $\text{TIMEDCEAEVALUATION}[C]$, a streaming evaluation algorithm with *f-update time* and output linear delay is a strong guarantee for efficiency, and it is the standard notion used in previous work [9, 11, 30]. Note that if one considers that $\mathcal{T}$ is small or constant (i.e., data complexity), the update time per event is proportional to the next event. Further, output-linear delay (see also constant-delay [33]) is the gold standard for enumeration problems, where the delay between outputs only depends on the next one. As it is standard in the literature, for these algorithms we assume the usual RAM model with unit costs and logarithmic size registers (see e.g., [2, 20]).

A class of timed CEA that admits a streaming evaluation algorithm. The main open problem is to pinpoint a class C of timed CEA such that $\text{TIMEDCEAEVALUATION}[C]$ admits a streaming evaluation algorithm with *f-update time* and output-linear delay for some function f . This problem is already open even for time automata (see, e.g., [21]), where the output is true or false after each event. Here, we present a first class of deterministic timed CEA that admits a streaming evaluation, improving our understanding of the efficient evaluation of timed CEA.

For $\sim \in \{\leq, \geq\}$, we say that a timed CEA $\mathcal{T} = (Q, P, X, Z, \Delta, q_0, F)$ has *$\sim$ -clock conditions* if all clock conditions are of the form $\bigwedge_{z \in Z} z \sim c_z$ for some $Z \subseteq Z$ and $c_z \in \mathbb{Q}_{\geq 0}$. Further, we say that a timed CEA is *monotonic* if it either has $\leq$ -clock conditions or $\geq$ -clock conditions.

THEOREM 6.1. *Let C^* be the class of all deterministic monotonic timed CEA with a single-clock. Then, $\text{TIMEDCEAEVALUATION}[C^*]$ admits a streaming evaluation algorithm with $O(|\mathcal{T}|^2 \cdot |e|)$ -update time and output-linear enumeration, for a timed CEA $\mathcal{T} \in C^*$ and next event e of the input stream.*

PROOF SKETCH. The streaming evaluation algorithm of Theorem 6.1 is a non-trivial extension of the algorithm presented in [11] for evaluating CEL over a sliding window. We dedicate this proof sketch to explain the technical novelty of this algorithm with respect to [11]. Given space restrictions, we present all the technical details of the algorithm in [18].

First, Theorem 6.1 strictly generalizes the algorithm of [11]: it can manage a more expressive set of deterministic timed CEA. Indeed, although [11] does not use timed CEA, the queries in [11] are of the form $\langle \varphi \rangle_{\leq c}$ for a standard CEL formula φ . This formula can be converted into a deterministic monotonic timed CEA and, thus, can be evaluated with the algorithm of Theorem 6.1. Further, our class of time automata can deal with more expressive queries like Example 6.2 (below).

For managing multiple clock constraints, we extend the data structure of [11], called timed Enumerable Compact Set (tECS), by introducing two new nodes that we called *reset nodes* and *clock-check nodes*, which are in charge of the intermediate resets and checks during a run. These nodes allow the restriction of the enumeration to a subset of outputs that satisfy a reset or check condition. Adding these two classes of nodes requires adapting the conditions on the data structure and the enumeration procedure to ensure output-linear delay. More importantly, we rethink the operations over the data structure, like union or extend, to ensure that the conditions are satisfied for output-linear enumeration of the results. Specifically, the reset nodes and check nodes do not produce output during the enumeration. Then, we need to enforce that they are not stacked during operations over the data structure. For this purpose, we introduce a *gadget* that combines resets and checks, which can be simplified whenever two of them are combined.

Finally, we need to revisit the evaluation algorithm of [11] and the use of the so-called *union lists*, a secondary data structure for managing lists of nodes. We generalize the conditions and invariants maintained over states and union lists to include the resets and checks in the transitions. This is the main purpose of the monotonic condition over the timed CEA, which allows us to generalize the main algorithmic ideas in [11]. $\square$

Example 6.2. One can check that $\mathcal{T}_1$ is a monotonic timed CEA. Furthermore, it is determinizable (by Theorem 5.3), and the determinization has one clock and is monotonic. By Theorem 6.1, we can efficiently evaluate the formula φ_1 . Similarly for the formula φ_2 of Example 3.2.

We leave open to find a more general class of timed CEA that admits a streaming evaluation algorithm. To the best of our knowledge, the algorithm of Theorem 6.1 is the first streaming evaluation strategy that combines automata evaluation with general time constraints and efficient enumeration.

7 Conclusions and future work

This paper explored the expressibility and efficient evaluation of time constraints in complex event recognition. We showed that timed CEL has the same expressive power as timed CEA, meaning that the time operators suffice to write any CER query with time constraints. We also found a subset of timed CEL that is definable by deterministic timed CEA with two clocks, allowing the processing of the queries without removing duplicate runs. Finally, we constructed an algorithm to process monotonic timed CEAs with one clock efficiently, setting grounds for future real-life implementation of the framework.

This paper leaves several open problems regarding the efficient evaluation of timed CEL queries. First, it is an open question whether one can evaluate any timed CEL formula with f -update time and output-linear delay for some function f . Even the case of a single clock is open (i.e., without the monotonic restriction). In [21], the streaming evaluation of timed automata with a single clock was solved; however, it is unclear how to extend the approach to more clocks for boolean output or to single-clock timed CEA. Even the case of swg queries would be an interesting problem to study. Second, finding more expressible classes of determinizable timed CEL formulas is a relevant problem for these algorithms. Finally, it will be interesting to consider time restrictions together with other features of CEP, like correlation (i.e., joins).

Acknowledgements

This work was funded by ANID Fondecyt Regular project 1230935, and by ANID – Millennium Science Initiative Program – Code ICN17_002.

References

- [1] J. Agrawal, Y. Diao, D. Gyllstrom, and N. Immerman. Efficient pattern matching over event streams. In *SIGMOD*, pages 147–160, 2008.
- [2] A. V. Aho and J. E. Hopcroft. *The design and analysis of computer algorithms*. Pearson Education India, 1974.
- [3] R. Alur and D. L. Dill. A theory of timed automata. *Theor. Comput. Sci.*, 126(2):183–235, 1994.
- [4] R. Alur, L. Fix, and T. A. Henzinger. Event-clock automata: a determinizable class of timed automata. *Theoretical Computer Science*, 211(1):253–273, 1999.
- [5] E. Asarin, P. Caspi, and O. Maler. Timed regular expressions. *Journal of the ACM*, 49(2):172–206, 2002.
- [6] E. Asarin, O. Maler, A. Pnueli, and J. Sifakis. Controller synthesis for timed automata. *IFAC Proceedings Vols.*, 31(18):447–452, 1998.
- [7] G. Bagan. MSO queries on tree decomposable structures are computable with linear delay. In *CSL*, volume 4207, pages 167–181. Springer, 2006.
- [8] C. Baier, N. Bertrand, P. Bouyer, and T. Brihaye. When are timed automata determinizable? In *ICALP*, pages 43–54. Springer, 2009.

- [9] C. Berkholz, J. Keppeler, and N. Schweikardt. Answering conjunctive queries under updates. In *PODS*, pages 303–318. ACM, 2017.
- [10] J. Boubeta-Puig, M. Bravetti, L. Llana, and M. G. Merayo. Analysis of temporal complex events in sensor networks. *Journal of Information and Telecommunication*, 1(3):273–289, 2017.
- [11] M. Bucci, A. Grez, A. Quintana, C. Riveros, and S. Vansummeren. CORE: a complex event recognition engine. *VLDB*, 15(9):1951–1964, 2022.
- [12] S. Chakraborty, L. T. Phan, and P. Thiagarajan. Event count automata: A state-based model for stream processing systems. In *RTSS*, pages 12–pp. IEEE, 2005.
- [13] G. Cugola and A. Margara. Complex event processing with t-rex. *Journal of Systems and Software*, 85(8):1709–1728, 2012.
- [14] G. Cugola and A. Margara. Processing flows of information: From data stream to complex event processing. *ACM Computing Surveys (CSUR)*, 44(3):1–62, 2012.
- [15] A. J. Demers, J. Gehrke, M. Hong, M. Riedewald, and W. M. White. Towards expressive publish/subscribe systems. In *EDBT*, volume 3896, pages 627–644, 2006.
- [16] O. Finkel. Undecidable problems about timed automata. In *FORMATS*, pages 187–199. Springer, 2006.
- [17] F. Florenzano, C. Riveros, M. Ugarte, S. Vansummeren, and D. Vrgoc. Constant delay algorithms for regular document spanners. In *PODS*, pages 165–177. ACM, 2018.
- [18] J. Garcia and C. Riveros. Complex event recognition under time constraints: towards a formal framework for efficient query evaluation. *CoRR*, abs/2504.00723, 2025.
- [19] N. Giatrakos, E. Alevizos, A. Artikis, A. Deligiannakis, and M. N. Garofalakis. Complex event recognition in the big data era: a survey. *VLDB J.*, 29(1):313–352, 2020.
- [20] E. Grandjean and L. Jachiet. Which arithmetic operations can be performed in constant time in the RAM model with addition? *CoRR*, abs/2206.13851, 2022.
- [21] A. Grez, F. Mazowiecki, M. Pilipczuk, G. Puppis, and C. Riveros. Dynamic data structures for timed automata acceptance. *Algorithmica*, 84(11):3223–3245, 2022.
- [22] A. Grez, C. Riveros, M. Ugarte, and S. Vansummeren. A formal framework for complex event recognition. *ACM Trans. Database Syst.*, 46(4):16:1–16:49, 2021.
- [23] T. A. Henzinger, K. Lehtinen, and P. Totzke. History-deterministic timed automata. In *CONCUR*, volume 243, 2022.
- [24] S. Klees-Meißner, R. Sattler, M. L. Schmid, N. Schweikardt, and M. Weidlich. Discovering event queries from traces: Laying foundations for subsequence-queries with wildcards and gap-size constraints. In *ICDT*, volume 220 of *LIPICs*, pages 18:1–18:21, 2022.
- [25] R. Koymans. Specifying real-time properties with metric temporal logic. *Real-time systems*, 2(4):255–299, 1990.
- [26] G. Li and H.-A. Jacobsen. Composite subscriptions in content-based publish/subscribe systems. In *ACM/IFIP/USENIX Int’l Conf. on Distributed Systems Platforms and Open Distributed Processing*, pages 249–269. Springer, 2005.
- [27] F. Lorber, A. Rosenmann, D. Ničković, and B. K. Aichernig. Bounded determinization of timed automata with silent transitions. *Real-Time Systems*, 53:291–326, 2017.
- [28] D. C. Luckham and J. Vera. An event-based architecture definition language. *IEEE transactions on Software Engineering*, 21(9):717–734, 1995.
- [29] L. Ma, C. Lei, O. Poppe, and E. A. Rundensteiner. Gloria: Graph-based sharing optimizer for event trend aggregation. In *SIGMOD*, pages 1122–1135. ACM, 2022.
- [30] M. Muñoz and C. Riveros. Streaming enumeration on nested documents. In *ICDT*, volume 220 of *LIPICs*, pages 19:1–19:18, 2022.
- [31] O. Poppe, C. Lei, E. A. Rundensteiner, and D. Maier. GRETA: graph-based real-time event trend aggregation. *VLDB*, 11(1):80–92, 2017.
- [32] M. L. Schmid and N. Schweikardt. Spanner evaluation over slp-compressed documents. In *PODS*, pages 153–165. ACM, 2021.
- [33] L. Segoufin. Enumerating with constant delay the answers to a query. In *ICDT*, pages 10–20, 2013.
- [34] P. V. Suman, P. K. Pandya, S. N. Krishna, and L. Manasa. Timed automata with integer resets: Language inclusion and expressiveness. In *FORMATS*, volume 5215, pages 78–92, 2008.
- [35] K. Tangwongsan, M. Hirzel, and S. Schneider. In-order sliding-window aggregation in worst-case constant time. *VLDB J.*, 30(6):933–957, 2021.
- [36] M. Ugarte and S. Vansummeren. On the difference between complex event processing and dynamic query evaluation. In *AMW*, 2018.
- [37] J. Verwiese, P. M. Grulich, J. Traub, and V. Markl. Survey of window types for aggregation in stream processing systems. *The VLDB Journal*, 32(5):985–1011, 2023.

Received December 2024; revised February 2025; accepted March 2025